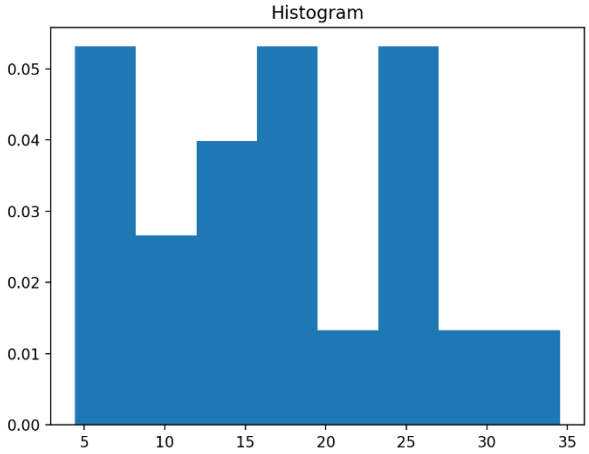
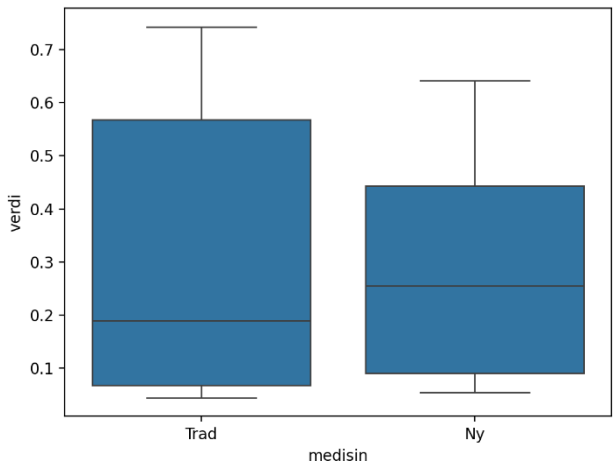
```python
#Oppgave 3B

import matplotlib.pyplot as plt
import numpy as np

data = np.array([7.98,10.82,15.88,17.00,24.22,12.20,8.17,16.53,7.46,
 14.31,34.55,19.46,20.21,13.58,10.98,4.42,24.92,30.29,
 23.45,23.36])

r_hat = np.mean(data)
print(r_hat)


plt.hist(data, bins=8, density= True)
plt.title("Histogram")
plt.show()

plt.boxplot(data)
plt.title("Boxplot")
plt.show()
```




```python
import numpy as np

#Oppgave 3E
def simuler_testobservator(n=20, a=5, r0=12.5):
    data_gamma = np.random.gamma(shape=a, scale=1/r0, size=n)

    r_hat_gamma = a / np.mean(data_gamma)

    Z = (r_hat_gamma - r0) /  (r0 / np.sqrt(n * a))

    return Z

#Lager en funksjon for å estimere med flere m, og konfidensintervall
def estimer_alpha_og_konfidensintervall(M=100000):
    Z_values = [simuler_testobservator() for i in range(M)]

    z_alpha = 1.2816

    #Finner antallet som skal forkastes
    forkast = [z > z_alpha for z in Z_values]

    #Estimator for alpha
    alpha_hat = np.mean(forkast)

    se = np.sqrt(alpha_hat * (1-alpha_hat) / M)

    lower = (alpha_hat - 1.96 * se).round(4) #Nedre grense for intervallet
    upper  = (alpha_hat + 1.96 * se).round(4) #Og øvre

    return alpha_hat.round(4), (lower, upper)

alpha_hat, konfidensintervall = estimer_alpha_og_konfidensintervall(M=100000)

print("alpha_hat er lik: ", alpha_hat) #Output = 0.1259
print("Konfidensintervall: ", konfidensintervall) #Output: {min, max} = {0.1238, 0.128}
```

```python
#Oppgave 4B

import numpy as np
from random import sample
import matplotlib.pyplot as plt

def testStatistic(x, nTrad):
    meanTrad = np.mean(x[0:(nTrad)])
    meanNew = np.mean(x[(nTrad):])
    return meanNew - meanTrad

x = [0.189,0.743,0.605,0.044,0.091,0.045,0.532,0.642,
 0.397,0.583,0.355,0.054,0.155,0.066,0.099]

statisticObserved = testStatistic(x,8)
print('Observert verdi: ',statisticObserved.round(4)) #Output: -0.1172

M = 10000

simulated = []

for i in range(M):
    xPermuted = sample(x, len(x))
    statisticSimulated = testStatistic(xPermuted, 8)
    simulated.append(statisticSimulated)

p_value = np.mean([s <= statisticObserved for s in simulated])

print('Simulert verdi: ', statisticSimulated.round(4)) #Output: 0.1726
print("p-value: ", p_value) #Output: 0.1905

plt.hist(simulated, bins=50)
plt.axvline(statisticObserved, color='red')
plt.show()
```

```python
#Oppgave 4A

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

x = [0.189,0.743,0.605,0.044,0.091,0.045,0.532,0.642,
 0.397,0.583,0.355,0.054,0.155,0.066,0.099]

data = pd.DataFrame({
    'verdi': x,
    'medisin': ['Trad', 'Trad', 'Trad', 'Trad', 'Trad', 'Trad', 'Trad',
                'Ny', 'Ny', 'Ny', 'Ny', 'Ny', 'Ny', 'Ny', 'Ny']
})

sns.boxplot(x='medisin',y='verdi',data=data)

plt.show()

#Kommentar til oppgaven: medianen er lavere på gammel medisin, men det er også mer spredt.
#Snittet virker å være lavere på den  nye medisinen, og at de lfestemålinger er lavere.
#Vi kan derfor konkludere med at det er rimelig å anta at medisinen fungerer
```